



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA

Escuela Profesional de Ingeniería de Sistemas

PROYECTO

**GENERADOR DE BACKUP Y PRUEBA DE
RESTAURACIÓN**

Curso: Base de Datos II

Docente: Ing. Patrick Jose Cuadros Quiroga

Integrantes:

Iker Alberto Sierra Ruiz (2023077090)

Julio Samuel Cortez Mamani (2023077283)

**Tacna – Perú
2026**

Logo de Mi Empresa

Logo de mi Cliente

Sistema “*Generar backup y prueba de restauración*”
Informe de Factibilidad

Versión {1.0}

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	MPV	ELV	ARV	27/03/202	Versión Original

INDICE GENERAL

1.	Descripción del Proyecto.....	5
2.	Riesgos.....	7
3.	Análisis de la Situación actual	7
4.	Estudio de Factibilidad	13
4.1	Factibilidad Técnica	14
4.2	Factibilidad económica.....	15
4.3	Factibilidad Operativa	20
4.4	Factibilidad Legal.....	20
4.5	Factibilidad Social.....	21
4.6	Factibilidad Ambiental	¡Error! Marcador no definido.
5.	Análisis Financiero.....	21
6.	Conclusiones.....	24

Informe de Factibilidad

1. Descripción del Proyecto

1.1 Nombre del proyecto

Generador de Backup y prueba de restauración.

1.2 Duración del proyecto

4 meses un ciclo académico

1.3 Descripción

En la actualidad, la pérdida de datos representa uno de los principales riesgos operativos para los desarrolladores. Disponer de un archivo de respaldo no garantizado por sí solo que dicho respaldo sea utilizable en una emergencia, ya que los volcados pueden corromperse o estar incompletos durante el proceso de transferencia o escritura.

Ante esta problemática surge SafeBridge , una aplicación de escritorio nativa (*Local-First*) orientada a desarrolladores individuales y pequeñas empresas para automatizar el ciclo de Copia de Seguridad (Backup) y su respectiva Validación de Integridad..A diferencia de un simple script de comandos, SafeBridge gestiona conexiones de bases de datos de manera segura y centralizada, orquesta las herramientas nativas de volcado (*pg_dump*, *mysqldump*, *sqlcmd*, *mongodump*), calcula hashes SHA-256 para validación de integridad, las firmas de terminación de los archivos (EOF) de manera nativa para comprobar que no hubo corrupción, y almacena un historial inmutable de registros locales de cada operación.

La solución está construida bajo los principios de Arquitectura Limpia utilizando tecnologías de última generación:

- I. Desktop Shell y Backend (Rust/Tauri v2): Ofrece un binario ligero, sin depender de un navegador pesado (como Electron), interactuando de forma segura con el sistema operativo mediante hilos asíncronos nativos.
- II. Base de Datos Embebida (SQLite): Mediante la librería rusqlite, proporciona una persistencia local estructurada, rápida y sin configuración para credenciales y registros de auditoría.
- III. Cifrado Local (AES-GCM): Cifra las contraseñas de las conexiones utilizando el estándar industrial aes-gcm de 256 bits en el backend de Rust.
- IV. Interfaz de Usuario (React + Tailwind CSS): Provee una experiencia de usuario moderna, técnica y reactiva, conectándose asíncronamente con los comandos de Tauri a través de un puente IPC (*Inter-Process Communication*).

1.4 Objetivos

1.4.1 Objetivo general

Desarrollar una solución de software de escritorio MVP nativa (Windows/Linux) que permita generar volcados (backups) de múltiples motores de base de datos y valide de forma automática la integridad del archivo resultante mediante verificación de firmas y generación de hash SHA-256, proporcionando al usuario una trazabilidad total mediante logs.

1.4.2 Objetivos Específicos

- Diseñar e implementar una arquitectura en Tauri v2 que desacople la interfaz web (React) de los procesos de sistemas nativos ejecutados en Rust.
- Implementar conexiones Multi-Motor (PostgreSQL, MySQL, SQL Server, MongoDB) garantizando que las contraseñas se almacenen cifradas mediante AES-GCM en SQLite.
- Integrar la generación de volcados mediante herramientas embebidas (sidecars en Tauri como pg_dump, mysqldump, etc.) evitando dependencias globales en el sistema operativo del usuario..
- Verificar nativamente la integridad de las copias de seguridad leyendo las firmas de terminación (EOF) del archivo y calculando su hash SHA-256 por fragmentos asíncronos.
- Desarrollar un panel (dashboard) analítico para consultar el ratio de éxito/fallo y revisar el historial de logs (tiempos de ejecución y rutas resultantes).
- Proponer una infraestructura en la nube teórica mediante Terraform , para futuras versiones, permitiendo el despliegue de almacenamiento S3 y entornos RDS de validación remota para entornos empresariales.

2. Riesgos

Categoría	Riesgo	Probabilidad	Estrategia de Mitigación Técnica

Técnico	Fallo en la invocación de sidecars nativos (ej. pg_dump, mysqldump).	Medios de comunicación	Validar en Rust la disponibilidad del sidecar antes de la ejecución; encapsular su uso mediante la API Commandde Tauri y capturar la salida de error (stderr) para el log de auditoría.
Técnico	Archivo .bako .sqlgenerado incompleto o corrupto.	Baja	Verificación nativa (EOF check) implementada en Rust para validar de inmediato las firmas de cierre (Dump completed onu análogas según el motor) sin

			cargar todo el archivo en la memoria RAM.
Técnico	Archivo .bako .sqlgenerado incompleto o corrupto.	Baja	SQLite es altamente confiable, pero se implementará manejo de fallos y resiliencia local usando directivas robustas como PRAGMA journal_mode = WAL;y un archivo único en el perfil protegido del usuario..
Técnico	Pérdida o corrupción de la base de datos local SQLite.	Baja	SQLite es altamente confiable, pero se implementará manejo de fallos y resiliencia

			local usando directivas robustas como PRAGMA journal_mode = WAL;y un archivo único en el perfil protegido del usuario..
Seguridad	Fuga de credenciales almacenadas localmente en la máquina.	Medios de comunicación	Uso estricto de cifrado simétrico AES-256-GCM para la columna passworden la base SQLite.Las claves se descifran en memoria RAM al vuelo y nunca se exponen ni envían al frontend web.
Operativo	Bloqueo o congelamiento de la	Baja	Tauri ejecuta los procesos

	UI durante la generación del respaldo.		pesados en hilos asíncronos (asynccomandos de Rust), emitiendo eventos de progreso continuo hacia React para mantener la interfaz fluida.
Humano	Abandono del proyecto por parte de un integrante del equipo.	Baja	Documentación continua del código, commits pequeños y frecuentes en GitHub, y gestión ágil de tareas con GitHub Projects.
Infraestructura	Errores en la definición teórica de recursos de Terraform Cloud.	Medios de comunicación	Mitigación preventiva utilizando comandos estrictos de

			validación sintáctica como terraform planterraform validate.
--	--	--	--

3. Análisis de la Situación actual

3.1 Planteamiento del problema

Los desarrolladores y administradores de bases de datos de aplicaciones pequeñas suelen generar copias de seguridad manualmente ejecutando comandos extensos en terminal o mediante clientes gráficos independientes que no validan los resultados. Un simple volcado puede verse interrumpido por pérdida de conexión de red, un fallo del servicio o disco lleno, generando un archivo corrupto que el usuario asume ciegamente como válido hasta que ocurre un desastre.

A demás, manejar conexiones hacia múltiples motores (PostgreSQL para un proyecto de microservicios, SQL Server para sistemas legados empresariales) implica usar herramientas disponibles, lo que fragmenta por completo el registro, la nomenclatura y la auditoría de cuándo y dónde se generaron las copias de seguridad.

SafeBridge MVP aborda este problema centralizando los comandos nativos de respaldo para 4 motores diferentes en una sola UI nativa, evaluando la salud del respaldo creado en disco inmediatamente después de su generación a través de un motor de inspección local, y brindando

un solo panel centralizado donde monitorear el estado real de las copias (Dashboard de operaciones).

3.2 Consideraciones de hardware y software

TIPO	Componente	Especificaciones Técnicas Evaluadas	Estado
HARDWARE	Equipos de desarrollo	Portátil/PC con procesador x64, 8 GB de RAM mínimo, almacenamiento SSD.	Disponible
	Interfaz de escritorio y backend	Rust 1.75+, Node.js (compilación), Tauri v2 CLI.	Disponible
SOFTWARE	Núcleo de interfaz	React 19, TypeScript, TailwindCSS v4.	Disponible
	Base de Datos Local	SQLite embebido (vía crate nativa rusqlite).	Disponible
	Motores de BD de Prueba	Imágenes locales de Docker (postgres, mysql, mssql, mongo) o XAMPP.	Disponible
	Control de Versiones	Git + GitHub (Gestión de ramas y CI/CD con GitHub Actions).	Disponible
	IaC (Extensión en la Nube)	Terraform v1.7+ (HashiCorp).	Disponible

4. Estudio de Factibilidad

El proyecto es técnicamente viable debido a que las tecnologías y librerías clave se encuentran disponibles en versiones estables y son de acceso abierto:

- **Desktop Framework:** Tauri está estabilizado en su versión 2.0. Su sistema de complementos (tauri-plugin-shell) permite encapsular e invocar los clientes de terminal de las BD (pg_dump.exe, mysqldump.exe, etc.) de forma aislada y segura.
- **Seguridad Criptográfica:** El ecosistema de Rust cuenta con cajas auditadas internacionalmente (aes-gcm, sha2, rand) para la criptografía de contraseñas y cálculo de hashes de archivos grandes mediante lectura en fragmentos asíncronos sin saturar la RAM del usuario.
- **Validación Rápida (MVP):** La factibilidad aumenta al basarse en validación EOF directa sobre disco en lugar de requerir levantar Docker Desktop en las máquinas de todos los usuarios para cada pequeña comprobación de archivo, ahorrando recursos de hardware.
- **Infraestructura como Código (IaC):** Terraform se define de forma teórica para su uso en la futura validación remota/cloud, asegurando que SafeBridge sea extensible a arquitecturas corporativas.

4.1 Factibilidad Técnica

El proyecto es técnicamente viable porque los recursos requeridos se encuentran disponibles y/o son de acceso libre para el equipo de desarrollo:

- Hardware: Los integrantes cuentan con equipos con especificaciones suficientes (Intel Core i5 o superior, 8 GB RAM, SSD) para ejecutar SQL Server y las herramientas de desarrollo.
- Software de base de datos: SQL Server 2022 Developer Edition incluye las operaciones necesadas (BACKUP DATABASE, RESTORE DATABASE).
- Lenguaje de programación: Python 3.12+ y librerías como pyodbc (conectividad), logging (auditoría) y customtkinter (interfaz gráfica) cubren todos los requisitos.
- Control de versiones: GitHub facilita el trabajo colaborativo y el control de cambios.
- Arquitectura: La aplicación se desarrollará bajo una arquitectura por capas, separando la lógica de negocio de la interfaz y de la capa de acceso a datos, lo que mejora la mantenibilidad.

La aplicación se conectará a SQL Server vía ODBC, y los archivos de backup se almacenarán en rutas definidas por el usuario.

4.2 Factibilidad Económica

Como aplicación de escritorio *Local-First*, SafeBridge no requiere servidores permanentes en su fase MVP, reduciendo a cero el costo operativo en licenciamiento de base de datos o alojamiento (el usuario almacena la copia de seguridad en su disco duro local o en un NAS).

4.2.1 Costos Generales

Categoría	Descripción	Costo Total Estimado (S/)
Costos Generales	Laptop de desarrollo, disco externo redundante, energía e internet..	800.00
Costos de Personal	2 desarrolladores junior (S/ 1,200.00 c/ux 4 meses).	9.600,00
Costos de Licenciamiento	Software de código abierto (Tauri, Rust, React, SQLite).	0.00
INVERSIÓN TOTAL		S/ 10.400,00

4.2.2 Análisis Económico e Infraestructura Cloud Efímera (Terraform)

Para cumplir con los requisitos técnicos corporativos a futuro y poder sincronizar los archivos generados localmente con un entorno seguro en la nube, se ha planificado una infraestructura basada en Amazon Web Services (AWS) gobernada totalmente por **Terraform**..Al gestionar esto con IaC, la base de datos de validación cruzada puede crearse y destruirse dinámicamente (terraform apply/ terraform destroy), lo que minimiza el costo frente a tener servidores encendidos 24/7.


```
# main.tf (Infraestructura teórica de validación cloud AWS)

terraform {

  required_version = ">= 1.7.0"

  required_providers {

    aws = {

      source = "hashicorp/aws"

      version = "~> 5.0"

    }

  }

}

resource "aws_s3_bucket" "safebridge_backups" {

  bucket = "safebridge-secure-backups-prd"

}

resource "aws_s3_bucket_versioning" "versioning_example" {

  bucket = aws_s3_bucket.safebridge_backups.id

  versioning_configuration {

    status = "Enabled"

  }

}

resource "aws_db_instance" "validation_rds" {

  allocated_storage = 20

  engine             = "postgres"
```

```

engine_version    = "15.4"

instance_class    = "db.t4g.micro"

username          = "admin_validator"

password          = "MutedSecurePassword2026_!"

skip_final_snapshot = true

}

```

Terraform de recursos	Tipo AWS	Uso Estimado / Métrica	Costo (S/) mes
Cubo S3	Almacenamiento estándar	50 GB / mes de retención.	S/ 4.50
RDS PostgreSQL	db.t4g.micro	40 horas/mes (encendido bajo demanda para verificar).	S/ 6.00
Transferencia	Salida a Internet	10 GB de descarga de logs.	S/ 3.50
NUBE MENSUAL TOTAL			S/ 14.00

4.2.3 Costos del ambiente

Concepto	Descripción	Costo (S/)
Licencia SQL Server	Versión Developer / Express	0.00
Repositorio GitHub	Plan Free (Privado)	0.00
Python & Librerías	Open Source (Software Libre)	0.00
Total		S/0.00

4.2.4 Costos de personal

Rol	Cantidad	Pago Mensual Estimado (S/)	Meses	Total
Desarrollador Backend / DBA	1	1,300.00	4	5,200.00
Desarrollador Frontend / QA	1	1,200.00	4	4,800.00
Total				S/ 10,000.00

4.2.5 Costos totales del desarrollo del sistema

Categoría	Monto (S/)
Costos Generales	300.00
Costos Operativos	560.00

Costos de Personal	10,000.00
TOTAL GENERAL	S/ 10,860.00

4.3 Factibilidad Operativa

El sistema es operativamente viable dado que resuelve directamente un proceso engorroso para el desarrollador individual:

- **Agrupación Múltiple Motores:**
Elimina la necesidad de memorizar las banderas y la sintaxis compleja de `pg_dump`, `mysqldump`, `sqlcmd` y `mongodump`.
- **Registro Centralizado:**
En lugar de dejar volcados sueltos en el disco, la BD SQLite embebida recuerda qué base de datos generó el archivo, con qué nombre, qué día, cuántos duros, su hash SHA-256, y si su firma era válida.
- **Desempeño Local:**
Al estar escrito en Rust, la aplicación ocupa mínimos MB de memoria RAM y no interfiere con el flujo de trabajo diario ni las herramientas del desarrollador.

4.4 Factibilidad Legal

- **Protección de datos personales:**

La aplicación opera 100% de manera local. Los archivos de bases de datos de clientes generados mediante backup no salen de la máquina del usuario (salvo que él opta por subirlos a su propia nube).

- **Licenciamiento Open Source:**

Los componentes core utilizan licencias altamente permisivas: React (MIT), Rust (MIT/Apache), SQLite (Dominio Público). La aplicación puede distribuirse gratuitamente o bajo licencia comercial sin restricciones legales.

4.5 Factibilidad Social y Ambiental

- **Impacto Social:**

Aumenta la madurez de ingeniería (DevOps) de desarrolladores individuales al introducirlos de forma visual al ciclo BCDR (Continuidad de Negocio).

- **Impacto Ambiental:**

Nulo para la aplicación en sí. Si se usa Terraform con AWS, la política "bajo demanda" (destruir tras usar) en lugar de "always-on" reduce el consumo de energía y la huella de carbono en los centros de datos.

5. Análisis Financiero

El análisis financiero asume la perspectiva de una pequeña firma de desarrollo de software o estudio independiente. Se evaluarán los beneficios de productividad generados por el uso de SafeBridge.

5.1 Justificación de la Inversión

5.1.1 Beneficios del Proyecto

Beneficios Cuantificables:

- Ahorro de Tiempo: Reducción del tiempo de generación manual y organización de volcados en al menos 2 horas-hombre por semana (2 horas $\times$ S/ 25.00/hora $\times$ 52 semanas = S/ 2,600.00 al año por ingeniero).
- Reducción de Riesgo (Prevención): Si asumimos un (1) incidente de pérdida crítica al año por un archivo corrupto que hubiera sido detectado a tiempo por SafeBridge, y un costo operativo/daño de negocio valuado conservadoramente en S/ 10,000 con un 50% de probabilidad de ocurrencia sin la herramienta, el beneficio por riesgo mitigado anual es S/ 5,000.00.

Beneficio total esperado: S/ 7,600.00 por desarrollador al año. Si consideramos una firma de desarrollo con tres (3) ingenieros, el beneficio sube a S/ 22,800.00 anuales.

5.1.2 Criterios de Inversión

Considerando una firma de desarrollo de 3 ingenieros, los indicadores financieros bajo una tasa de descuento (COK) del 10%:

Indicador Financiero	Año 0 (Inversión Inicial)	Año 1 (Beneficio Neto Anual)	Resultado y Decisión

Inversión Total	S/ -10.400,00		Costo de desarrollo personal.
Flujo de Caja		S/ 22.800,00	Ahorros y prevención de riesgos.
Relación Beneficio/Costo (B/C)			2.19
Valor Actual Neto (VAN)			S/ 10.327,27
Tasa Interna de Retorno (TIR)			119%

5.1.2.1 Relación Beneficio/Costo (B/C)

- $B/C = \text{Beneficio Anual} / \text{Costo Total} = S/ 17,400.00 / S/ 10,860.00 = 1.60$
- El B/C es mayor que 1, lo que indica que por cada sol invertido se obtienen S/ 1.60 en beneficios. Por lo tanto, el proyecto se ACEPTA bajo este criterio.

5.1.2.2 Valor Actual Neto (VAN)

- Se utiliza una tasa de descuento (COK) del 10%.
- $$VAN = -S/ 10,860.00 + (S/ 17,400.00 / 1.10) = S/ 4,958.18$$
- El VAN es mayor que cero, confirmando que el proyecto genera valor neto positivo. Por lo tanto, el proyecto se ACEPTA bajo este criterio.

5.1.2.3 Tasa Interna de Retorno (TIR)

- Se utiliza una tasa de descuento (COK) del 10%.
- $$VAN = -S/ 10,860.00 + (S/ 17,400.00 / 1.10) = S/ 4,958.18$$
- El VAN es mayor que cero, confirmando que el proyecto genera valor neto positivo. Por lo tanto, el proyecto se ACEPTA bajo este criterio.

6. Conclusiones

- Factibilidad Técnica — VIABLE: La arquitectura Rust + Tauri v2 + React es robusta, probada, moderna y sumamente eficiente para invocar sidecars nativos de bases de datos sin sobrecargar la RAM del cliente. Factibilidad Económica — VIABLE: el costo total estimado del proyecto asciende a S/ 10,860.00, concentrado principalmente en recurso humano. Las herramientas base son gratuitas.
- Factibilidad Económica — VIABLE: El desarrollo consume horas-hombre al estar basado en librerías Open Source permisivas. El

análisis B/C arroja un ratio superior a 2, validando el retorno masivo de la inversión para firmas de desarrollo mediante mitigación. La infraestructura simulada en Terraform es extremadamente económica. Factibilidad Legal — VIABLE: el proyecto se alinea con la Ley N.º 29733 de Protección de Datos Personales y utiliza software con licencias compatibles con el uso académico.

- Factibilidad Operativa — VIABLE: La interfaz gráfica es intuitiva y soluciona un problema directo del sector: la validación en tiempo real de un backup generado localmente y su almacenamiento en SQLite para auditorías e históricos.
- Factibilidad Legal/Ambiental — VIABLE: Uso exclusivo de licencias permisivas (MIT, Apache, Dominio Público), nulo procesamiento de información sensible fuera del entorno y reducción de la huella de carbono mediante el uso de entornos en la nube efímeros.

En conclusión, el proyecto "SafeBridge: Orquestador Multi-Motor de Respalos y Validación de Integridad" es un software totalmente viable y de alta calidad técnica que cumple con las demandas académicas y comerciales de la ingeniería de software.